

Netease API Documentation

For Frontend Developer

Version: 1.0.0

Base URL: `http://172.16.200.90:8000`

Interactive Docs: <http://172.16.200.90:8000/docs>

Authentication

Login

Endpoint: `POST /auth/login`

Request Body:

```
json

{
  "username": "string",
  "password": "string"
}
```

Response (200 OK):

```
json

{
  "access_token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",
  "token_type": "bearer",
  "role": "admin",
  "username": "sina"
}
```

Response (401 Unauthorized):

```
json

{
  "detail": "Incorrect username or password"
}
```

How to use the token:

javascript

```
// Store token
localStorage.setItem('token', response.data.access_token);

// Use in future requests (not implemented yet, but will be needed)
const token = localStorage.getItem('token');
axios.get('/api/users/', {
  headers: {
    'Authorization': `Bearer ${token}`
  }
});
```

User Management

1. Get All Users

Endpoint: `GET /api/users/`

Response (200 OK):

json

```
[
  {
    "id": 1,
    "username": "sina",
    "email": "bimesl@email.com",
    "role": "admin",
    "is_active": true
  },
  {
    "id": 2,
    "username": "test",
    "email": "test@example.com",
    "role": "user",
    "is_active": true
  }
]
```

2. Get Single User

Endpoint: GET /api/users/{user_id}

Example: GET /api/users/1

Response (200 OK):

```
json

{
  "id": 1,
  "username": "sina",
  "email": "bimesl@email.com",
  "role": "admin",
  "is_active": true,
  "created_at": "2024-11-20T10:30:45.123456"
}
```

Response (404 Not Found):

```
json

{
  "detail": "User with ID 999 not found"
}
```

3. Create User

Endpoint: POST /api/users/

Request Body:

```
json

{
  "username": "newuser",
  "email": "newuser@example.com",
  "password": "password123",
  "role": "user",
  "is_active": true
}
```

Field Validation:

- `username`: 3-50 characters, required
- `email`: Valid email format, required
- `password`: 4-72 characters, required
- `role`: One of: "admin", "manager", "user", "guest" (default: "user")
- `is_active`: boolean (default: true)

Response (201 Created):

```
json
{
  "id": 5,
  "username": "newuser",
  "email": "newuser@example.com",
  "role": "user",
  "is_active": true,
  "created_at": "2024-11-20T15:30:45.123456"
}
```

Response (400 Bad Request):

```
json
{
  "detail": "Username 'newuser' already exists"
}
```

4. Update User

Endpoint: `PUT /api/users/{user_id}`

All fields are optional - only send fields you want to update.

Request Body:

```
json
```

```
{
  "username": "updated_username",
  "email": "newemail@example.com",
  "password": "newpassword123",
  "role": "manager",
  "is_active": false
}
```

Example - Update only email:

```
json

{
  "email": "newemail@example.com"
}
```

Response (200 OK):

```
json

{
  "id": 5,
  "username": "updated_username",
  "email": "newemail@example.com",
  "role": "manager",
  "is_active": false,
  "created_at": "2024-11-20T15:30:45.123456"
}
```

Response (404 Not Found):

```
json

{
  "detail": "User with ID 999 not found"
}
```

5. Delete User

Endpoint: `DELETE /api/users/{user_id}`

Response (200 OK):

json

```
{
  "success": true,
  "message": "User 'testuser' deleted successfully"
}
```

Response (404 Not Found):

json

```
{
  "detail": "User with ID 999 not found"
}
```

6. Search Users

Endpoint: `GET /api/users/search/?q={search_term}`

Query Parameters:

- `q`: Search term (minimum 2 characters, searches in username and email)

Example: `GET /api/users/search/?q=admin`

Response (200 OK):

json

```
[
  {
    "id": 1,
    "username": "admin",
    "email": "admin@example.com",
    "role": "admin",
    "is_active": true
  }
]
```

Login Logs

1. Get All Logs

Endpoint: `GET /api/logs/`

Query Parameters (optional):

- `limit`: Number of records (default: 50, max: 100)
- `success_only`: Filter by success status (true/false)

Examples:

```
GET /api/logs/           # All logs (last 50)
GET /api/logs/?limit=100 # Last 100 logs
GET /api/logs/?success_only=true # Only successful logins
GET /api/logs/?success_only=false # Only failed logins
```

Response (200 OK):

```
json
[
  {
    "id": 10,
    "username": "sina",
    "success": true,
    "ip_address": "172.16.200.90",
    "user_agent": "Mozilla/5.0...",
    "message": "Login successfully with admin Role.",
    "timestamp": "2024-11-20T10:30:45.123456"
  },
  {
    "id": 9,
    "username": "test",
    "success": false,
    "ip_address": "192.168.1.100",
    "user_agent": "curl/7.68.0",
    "message": "Incorrect username or password",
    "timestamp": "2024-11-20T10:25:30.123456"
  }
]
```

2. Get User-Specific Logs

Endpoint: `GET /api/logs/user/{username}`

Query Parameters (optional):

- `limit`: Number of records (default: 20)

Example: `GET /api/logs/user/sina?limit=10`

Response: Same format as above

3. Get Login Statistics

Endpoint: `GET /api/logs/stats`

Response (200 OK):

```
json
{
  "total_attempts": 25,
  "successful_logins": 18,
  "failed_attempts": 7,
  "success_rate": 72.0,
  "recent_successful_logins": [
    {
      "username": "sina",
      "timestamp": "2024-11-20 10:30:45",
      "ip_address": "172.16.200.90"
    },
    {
      "username": "admin",
      "timestamp": "2024-11-20 10:25:30",
      "ip_address": "192.168.1.50"
    }
  ]
}
```



Health Check

Check API Status

Endpoint: `GET /health`

Response (200 OK):

```
json
{
  "status": "ok"
}
```

Error Responses

All error responses follow this format:

```
json
{
  "detail": "Error message here"
}
```

Common HTTP Status Codes:

- `200 OK` - Success
- `201 Created` - Resource created successfully
- `400 Bad Request` - Invalid input data
- `401 Unauthorized` - Authentication failed
- `403 Forbidden` - No permission
- `404 Not Found` - Resource not found
- `500 Internal Server Error` - Server error

Frontend Implementation Examples

Example: Login Flow

```
javascript
```

```
const API_BASE = 'http://172.16.200.90:8000';

async function login(username, password) {
  try {
    const response = await fetch(`${API_BASE}/auth/login`, {
      method: 'POST',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify({ username, password })
    });

    if (response.ok) {
      const data = await response.json();
      localStorage.setItem('token', data.access_token);
      localStorage.setItem('username', data.username);
      localStorage.setItem('role', data.role);
      return { success: true, data };
    } else {
      const error = await response.json();
      return { success: false, error: error.detail };
    }
  } catch (error) {
    return { success: false, error: 'Network error' };
  }
}
```

Example: Fetch Users with Error Handling

javascript

```
async function getUsers() {
  try {
    const response = await fetch(`${API_BASE}/api/users/`);

    if (!response.ok) {
      throw new Error(`HTTP error! status: ${response.status}`);
    }

    const users = await response.json();
    return users;
  } catch (error) {
    console.error('Error fetching users:', error);
    return [];
  }
}
```

Example: Create User

javascript

```
async function createUser(userData) {
  try {
    const response = await fetch(`${API_BASE}/api/users/`, {
      method: 'POST',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify(userData)
    });

    const data = await response.json();

    if (response.ok) {
      return { success: true, user: data };
    } else {
      return { success: false, error: data.detail };
    }
  } catch (error) {
    return { success: false, error: 'Network error' };
  }
}
```

Example: Update User

javascript

```
async function updateUser(userId, updates) {
  try {
    const response = await fetch(`${API_BASE}/api/users/${userId}`, {
      method: 'PUT',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify(updates)
    });

    const data = await response.json();

    if (response.ok) {
      return { success: true, user: data };
    } else {
      return { success: false, error: data.detail };
    }
  } catch (error) {
    return { success: false, error: 'Network error' };
  }
}
```

Example: Delete User

javascript

```
async function deleteUser(userId) {
  if (!confirm('Are you sure you want to delete this user?')) {
    return { success: false, cancelled: true };
  }

  try {
    const response = await fetch(`${API_BASE}/api/users/${userId}`, {
      method: 'DELETE'
    });

    const data = await response.json();

    if (response.ok) {
      return { success: true, message: data.message };
    } else {
      return { success: false, error: data.detail };
    }
  } catch (error) {
    return { success: false, error: 'Network error' };
  }
}
```

CORS Configuration

CORS is currently set to allow all origins (*) for development.

Note: In production, this should be restricted to specific domains.

Notes for Frontend Developer

1. **VPN Required:** The server is on a private network. VPN connection is required to access the API.
2. **Base URL:** Always use `http://172.16.200.90:8000` as the base URL.
3. **Content-Type:** Always set `Content-Type: application/json` for POST/PUT requests.
4. **Error Handling:** Always handle both network errors and API errors.
5. **Token Storage:** Store the JWT token in localStorage after login (will be needed for future protected routes).
6. **User Roles:**

- `admin`: Full access
- `manager`: Limited admin access
- `user`: Standard user access
- `guest`: Read-only access

7. **Testing:** Use `/docs` endpoint for interactive API testing before implementing in frontend.

8. **Test Accounts:**

- Username: `sina` / Password: `1234` (Admin)

Contact

Backend Developer: [Your Name]

Last Updated: November 20, 2024

Coming Soon

- Asset Management APIs
- Asset Discovery APIs
- CSV Import/Export
- Advanced filtering and pagination
- Real-time notifications